

ESE410034-41010086ME (20262) - Estatística para análise de dados

[Painel](#) / [Meus cursos](#) / [ESE410034-41010086ME \(20262\)](#) / [O Filtro de Kalman](#) / [Avaliação C1: Fusão Sensorial e Estimação de Pose...](#)

Avaliação C1: Fusão Sensorial e Estimação de Pose com Filtro de Kalman

Aberto: terça-feira, 30 jun. 2026, 00:00

Vencimento: segunda-feira, 10 ago. 2026, 23:59

Trabalho Prático: Fusão Sensorial e Estimação de Pose com Filtro de Kalman

1. Objetivo

O objetivo deste trabalho é aplicar os conceitos de inferência bayesiana e processos estocásticos através da implementação de um **Filtro de Kalman Linear (KF)**. Os alunos deverão realizar a fusão sensorial dos dados de uma Unidade de Medida Inercial (IMU) — combinando o Giroscópio e o Acelerômetro — para estimar a atitude (pose/angulação) de um corpo rígido de forma robusta ao ruído.

2. Contextualização Teórica (A ser desenvolvida no relatório)

No relatório descritivo, o grupo deverá demonstrar formalmente a relação matemática entre o **Teorema de Bayes** e o **Filtro de Kalman**.

- Demonstre como a etapa de *Predição* do filtro se correlaciona com a probabilidade *A Priori* (baseada no modelo dinâmico do giroscópio).
- Demonstre como a etapa de *Atualização* se correlaciona com a *Verossimilhança* (Likelihood) da medição do acelerômetro para gerar a distribuição *A Posteriori*.
- Apresente as equações matriciais completas do filtro que serão utilizadas no código.

3. Requisitos do Projeto

O trabalho está dividido em três etapas obrigatórias de desenvolvimento:

Parte A: O Filtro Genérico (Python)

Implementar uma classe genérica em Python para o Filtro de Kalman utilizando exclusivamente a biblioteca **numpy** para operações matriciais.

- A classe **não** deve ser fixa para o problema da IMU. Ela deve receber as matrizes estruturais (F , B , H , Q , R , P_0 e x_0) em seu construtor e possuir os métodos independentes `.predict(u)` e `.update(z)`.
- O objetivo desta etapa é avaliar a habilidade de traduzir equações algébricas lineares em código computacional limpo.

Parte B: Simulação e Validação

Utilizando a classe desenvolvida na Parte A, o grupo deverá criar um script de simulação computacional:

- Gerar um sinal sintético que represente a inclinação real (ex: uma onda senoidal).
- Simular o comportamento do **Giroscópio** (adicionando ruído branco gaussiano de baixa variância à derivada do sinal real).
- Simular o comportamento do **Acelerômetro** (adicionando ruído gaussiano de alta variância diretamente sobre o sinal real).
- Executar o filtro sobre os dados ruidosos, plotar os gráficos comparativos (Real vs. Medido vs. Filtrado) e calcular o Erro Quadrático Médio (EQM) para validar o ganho estatístico.

Parte C: Implementação Prática (Hardware ou Simulador)

Nesta etapa, o filtro deve rodar com dados dinâmicos de uma IMU real ou simulada individualmente por passo de tempo. O grupo pode escolher **uma** das seguintes abordagens:

- Opção 1 (Hardware Real):** Utilizar um microcontrolador (Arduino, ESP32, Raspberry Pi Pico, etc.) conectado a uma IMU (ex: MPU6050). O microcontrolador deve coletar as leituras brutas. O filtro de Kalman pode rodar diretamente embarcado (C/C++) ou os dados podem ser enviados via Serial em tempo real para o script Python processar e plotar em formato de *Dashboard* (ex: usando `matplotlib.animation` ou `dash`).
- Opção 2 (Simulador de Física/Robótica):** Utilizar um ambiente simulado que forneça telemetria de sensores inerciais passo a passo (ex: CoppeliaSim, Webots, ROS/Gazebo ou mesmo o simulador online Wokwi para programar o microcontrolador virtual). O filtro deve guiar a estimação da pose do robô/objeto simulado.

4. Entregáveis

Cada grupo deverá submeter um repositório (ex: GitHub) contendo:

- Relatório Técnico (PDF):** Contendo a fundamentação bayesiana, modelagem matemática do sistema (definição das matrizes para o problema da IMU), análise dos resultados gráficos e discussão sobre como os parâmetros Q e R foram sintonizados.

- 2. **Código-Fonte: Organização do Repositório conforme escrito abaixo**
- 3. **Vídeo de Demonstração:** Um vídeo curto (máx. 3 minutos) mostrando o funcionamento da Parte C (seja o hardware mexendo e o gráfico atualizando ou o simulador rodando).

5. Critérios de Avaliação

- **Fundamentação Estatística (25%):** Clareza e correção na dedução das equações e conexão com o Teorema de Bayes.
- **Qualidade da Arquitetura do Código (25%):** Generalidade da classe do Filtro de Kalman em Python e organização do código.
- **Análise de Simulação (20%):** Sintonização correta das matrizes de covariância (\$Q\$ e \$R\$) e avaliação estatística do erro.
- **Aplicação Prática/Simulada (30%):** Sucesso na aquisição e filtragem dos dados de pose em tempo real (Hardware ou Simulador).

Materiais auxiliares:

<https://www.youtube.com/watch?v=5HuN9iL-zxU>
<https://www.youtube.com/playlist?list=PLn8PRpmsu08ryYoBpEKzoMOveSTyS-h4a>

Requisitos de Organização do Repositório (GitLab UFSC / Códigos UFSC)

O código-fonte do projeto deve ser entregue obrigatoriamente através de um link para um repositório no **Github** ou **GitLab da UFSC (Códigos UFSC)**. O grupo deve garantir que as permissões do repositório estejam configuradas como **Público** ou que o professor/avaliador seja adicionado como membro com permissão de leitura.

O repositório será avaliado pela sua clareza e profissionalismo, devendo conter a seguinte estrutura:

- 1. **README.md (Página Inicial do Repositório):**
 - **Título e Descrição:** Nome do projeto e uma breve explicação do seu funcionamento (Fusão sensorial com KF).
 - **Identificação:** Nome completo e matrícula de todos os integrantes do grupo.
 - **Instruções de Instalação e Execução (Parte A e B):** Passo a passo de como clonar o repositório do GitLab UFSC, instalar as dependências e executar o script de simulação em Python.
 - **Guia de Execução/Montagem (Parte C):** * *Se escolheram Hardware:* Esquema de ligação dos pinos da IMU no microcontrolador e instruções para carregar o firmware.
 - *Se escolheram Simulador:* Links ou instruções de como configurar o ambiente virtual (CoppeliaSim, Webots, Wokwi, etc.) para rodar o projeto.
- 2. **Gerenciamento de Dependências:**
 - Inclusão de um arquivo `requirements.txt` na raiz do projeto contendo todas as bibliotecas Python utilizadas (ex: `numpy`, `matplotlib`, `pyserial`), permitindo a instalação rápida via `pip install -r requirements.txt`.
- 3. **Organização de Pastas Sugerida:**

Plaintext

```
| /src                # Código-fonte principal
| | kalman.py         # Classe genérica do Filtro de Kalman
| | simulacao.py      # Script da Parte B (Dados sintéticos)
| | main_pratica.py   # Script da Parte C (Interface com Hardware/Simulador)
| /firmware           # Código do microcontrolador (se aplicável, ex: .ino)
| /docs               # Relatório Técnico em PDF e esquemas de circuito
| requirements.txt    # Dependências do Python
| README.md           # Instruções gerais (Markdown)
```

- 4. **Boas Práticas de Versionamento e Código:**
 - Código devidamente **comentado**, detalhando a lógica de montagem das matrizes estocásticas.
 - Histórico de *commits* limpo, demonstrando a divisão de tarefas e a evolução do trabalho entre os membros do grupo.
 - Esta nota será contabilizado como C1, ou seja, será duplicada para:
 - **Teste Online 1 e Trabalho Prático 1**

Adicionar envio

Status de envio

Status de envio	Nenhum envio foi feito ainda
Status da avaliação	Não há notas

Tempo restante	35 dias 14 horas restando
Última modificação	-
Comentários sobre o envio	▶ Comentários (0)

[◀ Serie do Mathworks - Filtro de Kalman](#)

Seguir para...

 Contate o suporte do site 

Você acessou como Adriano Pertile (202601914) (Sair)
ESE410034-41010086ME (20262)

Moodle UFSC - Presencial